All searchable database encryption leaks some information about the encrypted values, and a patient attacker may eventually be able to defeat any such scheme. For this reason, and the complexity, I recommend that developers concentrate on basic database access controls before investigating more complex solutions. You should still enable built-in database encryption if your database storage is hosted by a cloud provider or other third party, and you should always encrypt all database backups—many backup tools can do this for you.

For readers that want to learn more, I've provided a heavily-commented version of the `DatabaseTokenStore` providing encryption and authentication of all token attributes, as well as *blind indexing* of usernames in a branch of the Git repository that accompanies the book (https://github.com/NeilMadden/apisecurityinaction/blob/database_encryption/natter-api/src/main/java/com/manning/apisecurityinaction/token/DatabaseTokenStore.java).

The main threat to your token database is through injection attacks or logic errors in the API itself that allow a user to perform actions against the database that they should not be allowed to perform. This might be reading other users' tokens or altering or deleting them. As discussed in chapter 2, use of prepared statements makes injection attacks much less likely. You reduced the risk even further in that chapter by using a database account with fewer permissions rather than the default administrator account. You can take this approach further to reduce the ability of attackers to exploit weaknesses in your database storage, with two additional refinements:

- You can use separate database accounts to perform destructive operations such as bulk deletion of expired tokens and deny those privileges to the database user used for running queries in response to API requests. An attacker that exploits an injection attack against the API is then much more limited in the damage they can perform. This split of database privileges into